

Chapter 13

Graph Theory-Based Portfolios



In this chapter we are going to explain how to incorporate the information obtained from graphs and dendrograms into the portfolio optimization process. Graphs have gained the attention of academics and practitioners since the seminal works of Mantegna (1999) and Tumminello et al. (2005) that show how the most representative relationships among assets can be represented through graphs like the minimum spanning tree and the triangulated maximally filtered graph, respectively. Graphs can be used to increase the diversification of investment portfolios through the addition of constraints on the absolute and relative position of the assets in the graph. On the other hand, dendrograms allow us to identify assets with similar behavior and we can use this information to diversify investment portfolios through the addition of constraints to invest only in assets that belong to different clusters.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import cvxpy as cp
import riskfolio as rp
import warnings

warnings.filterwarnings("ignore")
pd.options.display.float_format = '{:.4%}'.format

# Date range
start = '2019-01-01'
end = '2023-12-30'

# Tickers of assets
assets = ['APA', 'BA', 'BAX', 'BMY', 'CMCSA',
          'CNP', 'CPB', 'DE', 'HPQ', 'JCI']
assets.sort()
```

```
# Downloading data
prices = yf.download(assets, start = start, end = end,
                    auto_adjust = False)
prices = prices.loc[:, ('Adj Close', slice(None))]
prices.columns = assets

# Calculating returns of Assets
returns = prices[assets].pct_change().dropna()
R = returns.to_numpy()
```

Listing 13.1 Data used in examples

13.1 Constraints on Graphs

13.1.1 Graphical Representation of Graphs

A graph is defined as the pair $G = (V, E)$, where V is a set of elements called vertices or nodes and E is a set of paired nodes called edges or links that represent the links between nodes.

1. **Undirected Graph** is a graph in which the edges do not point in any direction; this means that the edges are bidirectional; for example, see Fig. 13.1.
2. **Directed Graph** is a graph in which the edges point a direction; this means that the edges are unidirectional; for example, see Fig. 13.2.
3. **Connected Graph** is a graph in which there is always a path from a node to any other node; for example, see Fig. 13.3.

Fig. 13.1 Undirected graph

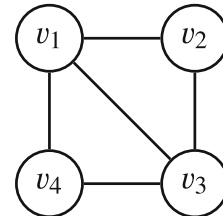


Fig. 13.2 Directed graph

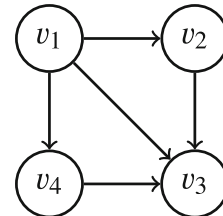
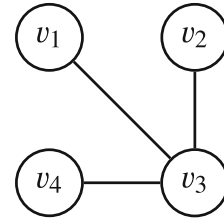
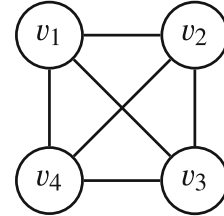
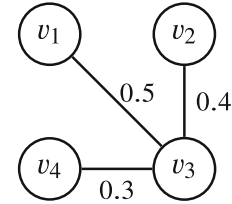
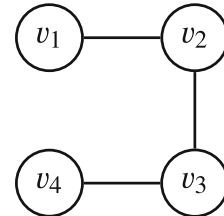
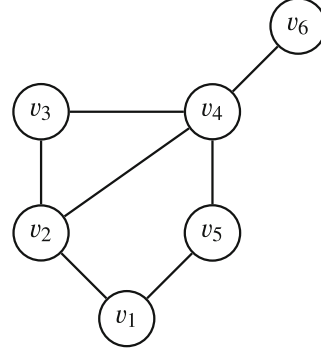


Fig. 13.3 Connected graph**Fig. 13.4** Complete graph**Fig. 13.5** Weighted graph**Fig. 13.6** Spanning tree

4. **Complete Graph** is an undirected graph in which every pair of distinct nodes is connected by a unique edge; for example, see Fig. 13.4.
5. **Weighted Graph** is a graph in which every edge has a weight; for example, see Fig. 13.5.
6. **Spanning tree** is a subgraph of an undirected connected graph, which includes all the nodes of the graph with a minimum possible number of edges; for example, see Fig. 13.6. The edges may or may not have weights assigned to them. The total number of spanning trees with n nodes that can be created from a complete unweighted graph is equal to n^{n-2} .

Fig. 13.7 Labeled graph**Fig. 13.8** Adjacency matrix A

$$\begin{array}{c}
 v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \\
 \begin{array}{l} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{array} \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}
 \end{array}$$

13.1.2 Matrix Representation of Graphs

The graphical representation is not helpful to pose optimization problems. To incorporate the information of graphs into optimization problems, we need a matrix representation of a graph. An adjacency matrix A is a matrix whose rows and columns represent the nodes of the graph, and the numbers in the matrix are the number of edges that connect node v_i to node v_j . For example, if we have the graph $G = (V, E)$ where $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ and $E = \{(v_1, v_2), (v_1, v_5), (v_2, v_3), (v_2, v_4), (v_3, v_4), (v_4, v_5), (v_4, v_6), (v_5, v_6)\}$, we can represent it in graphical and matrix form as shown in Figs. 13.7 and 13.8, respectively.

One important property of the adjacency matrix A is that the i, j -th element of the matrix A^k ¹ gives us the number of walks or connections of size k between node v_i and v_j . Based on this property, we can define, for a graph without loops of n nodes, the connection matrix $\mathbf{B}_k$ of nonclosed walks of length k as follows:

$$\mathbf{B}_k = \mathbf{1}_{x \geq 1}(A^k + I_n) - I_n \quad (13.1)$$

where $\mathbf{1}_{x \geq 1}(\cdot)$ is the element-wise indicator function for values $x \geq 1$ and I_n is the identity matrix of size n . The matrix $\mathbf{B}_k$ tells us if two different nodes v_i and v_j are connected through a walk of size k .

¹ $A^k = \underbrace{A A \dots A}_{k \text{ times}}$.

Fig. 13.9 Matrix A^2

$$\begin{array}{c}
 v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \\
 \begin{array}{l}
 v_1 \\
 v_2 \\
 v_3 \\
 v_4 \\
 v_5 \\
 v_6
 \end{array}
 \begin{bmatrix}
 2 & 0 & 1 & 2 & 0 & 0 \\
 0 & 3 & 1 & 1 & 2 & 1 \\
 1 & 1 & 2 & 1 & 1 & 1 \\
 2 & 1 & 1 & 4 & 0 & 0 \\
 0 & 2 & 1 & 0 & 2 & 1 \\
 0 & 1 & 1 & 0 & 1 & 1
 \end{bmatrix}
 \end{array}$$

Fig. 13.10 Connection matrix $\mathbf{B}_2$

$$\begin{array}{c}
 v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \\
 \begin{array}{l}
 v_1 \\
 v_2 \\
 v_3 \\
 v_4 \\
 v_5 \\
 v_6
 \end{array}
 \begin{bmatrix}
 0 & 0 & 1 & 1 & 0 & 0 \\
 0 & 0 & 1 & 1 & 1 & 1 \\
 1 & 1 & 0 & 1 & 1 & 1 \\
 1 & 1 & 1 & 0 & 0 & 0 \\
 0 & 1 & 1 & 0 & 0 & 1 \\
 0 & 1 & 1 & 0 & 1 & 0
 \end{bmatrix}
 \end{array}$$

Fig. 13.11 Range connection matrix $\mathbf{B}_{1,2}$

$$\begin{array}{c}
 v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \\
 \begin{array}{l}
 v_1 \\
 v_2 \\
 v_3 \\
 v_4 \\
 v_5 \\
 v_6
 \end{array}
 \begin{bmatrix}
 0 & 1 & 1 & 1 & 1 & 0 \\
 1 & 0 & 1 & 1 & 1 & 1 \\
 1 & 1 & 0 & 1 & 1 & 1 \\
 1 & 1 & 1 & 0 & 1 & 1 \\
 1 & 1 & 1 & 1 & 0 & 1 \\
 0 & 1 & 1 & 1 & 1 & 0
 \end{bmatrix}
 \end{array}$$

In Fig. 13.9 the matrix A^2 tells us the number of walks of length 2 that connect nodes v_i and v_j ; and in Fig. 13.10 the connection matrix $\mathbf{B}_2$ tells us if two different nodes v_i and v_j are connected through at least one walk of length 2. We can generalize the idea for nodes that are connected through walks of lengths between 1 and l through the range connection matrix $\mathbf{B}_{1,l}$ that formally can be expressed as:

$$\mathbf{B}_{1,l} = \mathbf{1}_{x \geq 1} \left(\sum_{k=1}^l \mathbf{B}_k \right) \quad (13.2)$$

In Fig. 13.11 the matrix $\mathbf{B}_{1,2}$ tells us if nodes v_i and v_j are connected by at least one walk of length 1 or 2.

Python code 13.2 shows how to calculate the power of the adjacency matrix, the connection matrix, and the range connection matrix.

```

# Adjacency matrix
A = np.array([[0,1,0,0,1,0],
              [1,0,1,1,0,0],
              [0,1,0,1,0,0],
              [0,1,1,0,1,1],
              [1,0,0,1,0,0],
              [0,0,0,1,0,0]])

# Power of Adjacency matrix
A_2 = np.linalg.matrix_power(A, 2)

# Connection matrix of walks of length 2
I_n = np.identity(A.shape[0])
B_2 = np.where(A_2 + I_n >= 1, 1, 0) - I_n

# Range connection matrix of walks of length lower than 2
walk_size = 2
B_1_2 = 0
for i in range(1, walk_size + 1):
    B_1_2 += np.where(np.linalg.matrix_power(A, i) + I_n >= 1,
                      1, 0) - I_n
B_1_2 = np.where(B_1_2 >= 1, 1, 0)

```

Listing 13.2 Matrix representation of a graph

13.1.3 Centrality Measures of Graphs

The matrix representation of a graph through the adjacency matrix tells us how nodes are linked. However, **we need some measures that tell us if a node is in the periphery or in the center of a graph**. To answer this question we use the centrality measures;² this kind of measures gives us an idea of the influence of a node in the network. Some popular centrality measures are:

13.1.3.1 Degree of a Node

The degree of a node is the number of edges that are directly connected to a node. We can calculate the degree vector $\mathbf{D}_n$, which is a vector that contains the degrees of all nodes, using the following formula:

$$\mathbf{D}_n = A \mathbf{1}_n \quad (13.3)$$

² A more extensive explanation of centrality measures can be found in Chapter 7 of Estrada (2011).

Fig. 13.12 Degree vector

$$\begin{array}{c} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{array} \begin{array}{c} v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \\ \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{array} \begin{array}{c} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \\ = \\ \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} \end{array} \begin{array}{c} \begin{bmatrix} 2 \\ 3 \\ 2 \\ 4 \\ 2 \\ 1 \end{bmatrix} \end{array}$$

Fig. 13.13 Eigenvector centrality vector

$$\begin{array}{c} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{array} \begin{array}{c} v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \\ \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{array} \begin{array}{c} \begin{bmatrix} 0.323 \\ 0.498 \\ 0.408 \\ 0.563 \\ 0.341 \\ 0.217 \end{bmatrix} \\ = \\ \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} \end{array} \begin{array}{c} \begin{bmatrix} 0.323 \\ 0.498 \\ 0.408 \\ 0.563 \\ 0.341 \\ 0.217 \end{bmatrix} \end{array}$$

$\frac{1}{2.599}$

where $\mathbf{1}_n$ is a column vector of ones of size $n \times 1$. For the adjacency matrix A in Fig. 13.7, the degree vector is calculated as shown in Fig. 13.12.

Python code 13.3 shows how to calculate the degree vector.

```
ones = np.ones((A.shape[0], 1))
D = A @ ones
```

Listing 13.3 Degree vector code

Only the top eigenvalue's eigenvector (Perron–Frobenius theorem, guaranteed because A has no negative entries) comes out all one sign, which is the only one that can be read as "everyone's importance, low to high."

13.1.3.2 Eigenvector Centrality

The eigenvector centrality of a node measures the influence of the node beyond its neighbors; this means that it considers connections that are composed of more than one edge (walks of length longer than 1). We can calculate the eigenvector centrality vector $\mathbf{EC}_n$, which is a vector that contains the eigenvector centralities of all nodes, using the following formula:

$$\mathbf{EC}_n = \frac{1}{\lambda_{\max}} A q_{\max} \quad (13.4)$$

where $\lambda_{\max}$ and $q_{\max}$ are the maximum eigenvalue and associated eigenvector of adjacency matrix A . For the adjacency matrix in Fig. 13.7, the eigenvector centrality vector is calculated as shown in Fig. 13.13.

Python code 13.4 shows how to calculate the eigenvector centrality vector.

```

# Eigenvector Centrality with Networkx
G = nx.from_numpy_array(A)
EC = nx.eigenvector_centrality_numpy(G)
EC = pd.Series(EC).to_numpy().reshape(-1, 1)

# Eigenvector Centrality with Numpy
w, v = np.linalg.eig(A)
k = np.argmax(w)
EC_1 = 1/w[k] * A @ v[:,k:k+1]

```

Listing 13.4 Eigenvector centrality vector code

13.1.3.3 Subgraph Centrality

The subgraph centrality measures the importance of a node in a graph by considering its participation in all closed walks starting (and ending) at it. We can calculate the subgraph centrality vector $\mathbf{EE}_n$, which is a vector that contains the subgraph centralities of all nodes, using the following formula:

$$\begin{aligned}\mathbf{EE}_n &= \text{diag}\left(e^A\right) \\ \mathbf{EE}_n &= \text{diag}\left(\sum_{i=0}^{\infty} \frac{A^i}{i!}\right)\end{aligned}\tag{13.5}$$

where $\text{diag}(\cdot)$ is the diagonalization operator and e^A is the matrix exponential of adjacency matrix A . The sum of all elements of subgraph centrality vector is the Estrada Index.³ For the adjacency matrix A in Fig. 13.7, the subgraph centrality vector is calculated as shown in Fig. 13.14.

Python code 13.5 shows how to calculate the subgraph centrality vector.

Fig. 13.14 Subgraph centrality vector

$$\begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{matrix} \begin{bmatrix} 2.496 \\ 3.851 \\ 2.971 \\ 4.678 \\ 2.549 \\ 1.722 \end{bmatrix}$$

³ The Estrada Index was originally designed as a way of characterizing the degree of folding of proteins.

```
# Subgraph Centrality with Networkx
G = nx.from_numpy_array(A)
SC = nx.subgraph centrality(G)
SC = pd.Series(SC).to_numpy().reshape(-1, 1)
```

Listing 13.5 Subgraph centrality vector code

13.1.4 Graphs in Financial Markets

13.1.4.1 Minimum Spanning Tree

The minimum spanning tree (MST) is a spanning tree of an undirected connected weighted graph in which the sum of the weights of the edges is as minimum as possible. This means that the MST in the case of codependence-based distances only retains the information of the largest codependence between two assets. Mantegna (1999) was the first to suggest that the relationships among assets can be represented using a MST. To construct a MST from a codependence matrix, we have to transform the codependence matrix into a distance or dissimilarity matrix.⁴ Then, based on the distance matrix, we can calculate the MST using one of the algorithms available.⁵

Python code 13.6 shows how to create and plot the MST with the data downloaded using Python code 13.1, and Fig. 13.15 shows the graphical representation of the MST.

```
fig, ax = plt.subplots(figsize=(8,5))
ax = rp.plot_network(returns=returns,
                    codependence="pearson",
                    linkage="ward",
                    kind='kamada',
                    title=' ',
                    ax=ax)
```

Listing 13.6 MST with Riskfolio-Lib

The MST provides useful information about the relationships among assets. Nodes in the border or periphery of the MST have only one edge (degree 1), and this number increases for nodes that are nearest to the center of the MST (degree higher than 1). The assets with lower degree are less related to others, while assets with higher degree are more related to others. We can create a more diversified portfolio if we select assets that are near to the periphery of the MST or assets that are not directly connected in the MST.

⁴ For more information see Chap. 7.

⁵ Networkx Python library has the function `minimum_spanning_tree` that allows us to calculate the MST using Boruvka, Prim and Kruskal algorithms.

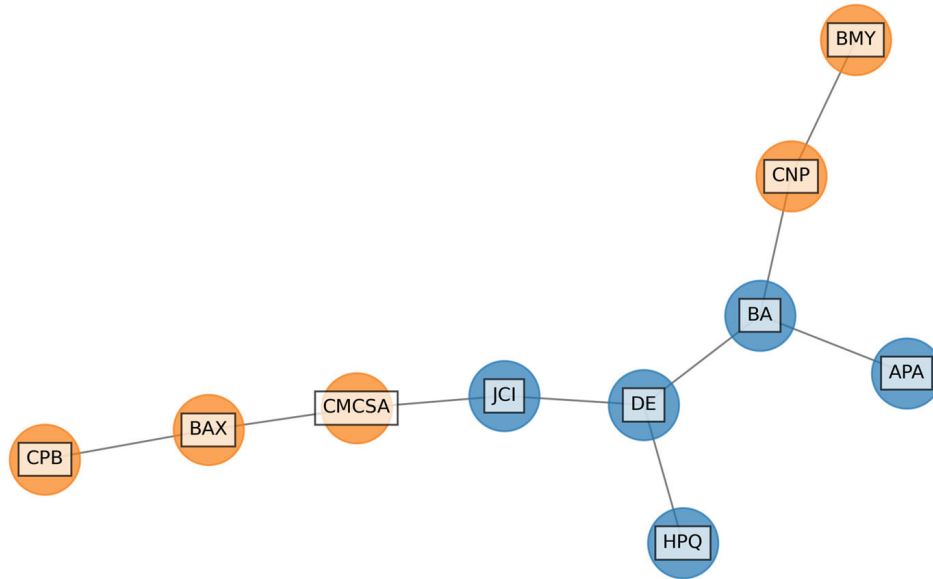


Fig. 13.15 MST of data sample

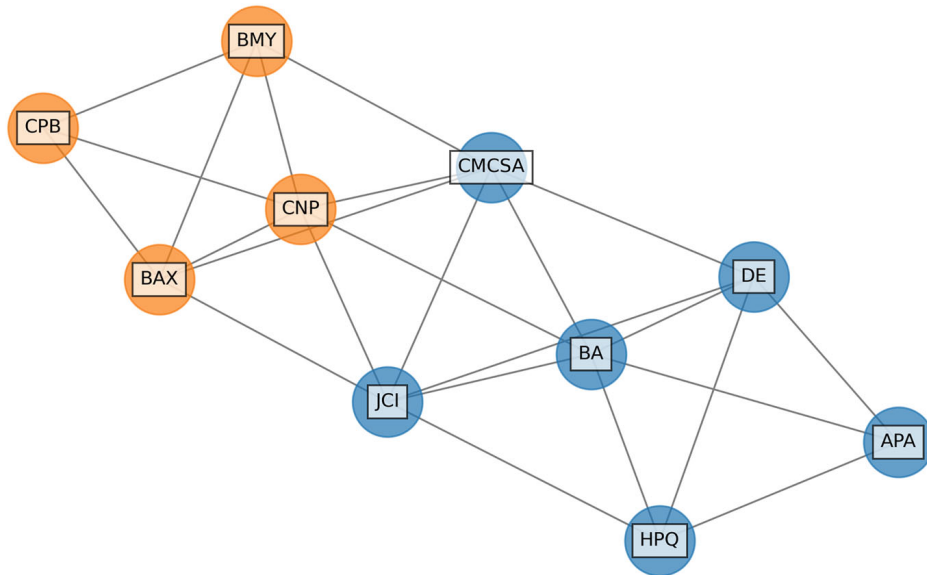
13.1.4.2 Triangulated Maximally Filtered Graph

The planar maximally filtered graph (PMFG) is a network-filtering method proposed by Tumminello et al. (2005) that provides a greedy solution to the weighted maximal planar graph (WMPG) problem. The WMPG consists in finding the planar subgraph that is maximal⁶ in which the sum of the weights of the edges is as minimum as possible. This means that the PMFG in the case of codependence-based distances retains the information of the largest codependences among assets while constraining that the subgraph to be globally a planar graph. The main disadvantage of the PMFG is that it is computationally costly; however, Massara et al. (2016) developed the triangulated maximally filtered graph (TMFG) that is a computationally efficient approximation to the WMPG problem and consists in building a triangulation that maximizes a score function associated with the amount of information retained by the network. To construct a TMFG we need a nonnegative codependence matrix; in case the original codependence matrix admits positive and negative values, we have to transform it into a nonnegative codependence matrix.⁷

⁶ We cannot add more edges without destroying planarity.

⁷ One suggestion is to transform the distance matrix into a codependence matrix using the formula $\rho_{i,j} = 1 - d_{i,j}^2$.

```
fig, ax = plt.subplots(figsize=(8,5))
ax = rp.plot_network(returns=returns,
                    codependence="pearson",
                    linkage="DBHT",
                    kind='kamada',
                    title='',
                    ax=ax)
```

Listing 13.7 TMFG with Riskfolio-Lib**Fig. 13.16** TMFG of data sample

Then, based on the nonnegative codependence matrix, we can calculate the TMFG using the TMFG algorithm.⁸

Python code 13.7 shows how to create and plot the TMFG with the data downloaded using Python code 13.1, and Fig. 13.16 shows the graphical representation of the TMFG.

The TMFG retains more information than the MST because TMFG has $3n - 6$ edges, while the MST only has $n - 1$ edges. Nodes in the border or periphery of the TMFG have three edges (degree three), and this number increases for nodes that are nearest to the center of the TMFG (degree higher than three). The assets with lower degree are less related to others, while assets with higher degree are more related to others. The same diversification criteria used with the MST apply to the TMFG.

⁸ Riskfolio-Lib Python library has the function `PMFG_T2s` to calculate the TMFG based on the nonnegative codependence matrix.

13.1.5 Graph Measures for Portfolios

13.1.5.1 Average Centrality Measures

In the case of an investment portfolio, we can define the average centrality measure of a portfolio using the following formula:

$$\mathbf{CM}(x) = \mathbf{C}_n' x \quad (13.6)$$

where x is the column vector of weights of the portfolio of size $n \times 1$ and $\mathbf{C}_n$ is a centrality measure vector. Formula 13.6 has the advantage that allows us to measure the diversification of a portfolio based on the influence of the assets in the graph.

Python code 13.8 shows how to calculate the average degree of the equally weighted portfolio.

```
# Equally weighted portfolio
T, n = R.shape
weights = np.ones((n,1))/n

# Degree Vector based on MST and Pearson correlation
C_v = rp.centralty_vector(returns,
    measure='Degree', # We can change the centrality measure
    codependence="pearson",
    graph="MST", # We can select the MST or TMFG
)

# Average degree of portfolio
CM = C_v @ weights
```

Listing 13.8 Average centrality measure with Riskfolio-Lib

13.1.5.2 Percentage Invested in Connected Assets

In the case of an investment portfolio, we can define the percentage invested in connected assets of a portfolio using the following formula:

$$\mathbf{CA}(x) = \frac{\mathbf{1}_n' (\mathbf{B}_{1,l} \odot |xx'|) \mathbf{1}_n}{\mathbf{1}_n' |xx'| \mathbf{1}_n} \quad (13.7)$$

where x is the column vector of weights of the portfolio of size $n \times 1$, $\odot$ is the Hadamard product or element-wise product, and $|\cdot|$ is the element-wise absolute value. Formula 13.7 allows us to measure the percentage invested in assets that are connected directly or through walks specified in the range connection matrix $\mathbf{B}_{1,l}$. The diversification idea is to invest in assets that are not connected; this means that when we invest in assets that are not connected, the absolute value of the product of

their weights is 0 ($|x_i x_j| = 0$) because one of the assets must have a weight equal to 0 ($x_i = 0$ or $x_j = 0$).

Python code 13.9 shows how to calculate the percentage invested in connected assets of the equally weighted portfolio.

```
# Equally weighted portfolio
T, n = R.shape
weights = np.ones((n,1))/n

# Percentage Invested in Connected Assets based on MST
# and Pearson correlation with walk size of length 1
CA = rp.connected_assets(returns,
    w=weights,
    codependence="pearson",
    graph="MST", # We can select the MST or TMFG
    walk_size=1)
```

Listing 13.9 Percentage invested in connected assets with Riskfolio-Lib

13.1.6 Centrality Measure Constraint

We can diversify a portfolio considering the influence of assets in the graph, just adding a constraint on an average centrality measure of a portfolio. We can pose this constraint as follows:

$$\begin{aligned} \text{opt}_x \quad & \phi(x) \\ \text{s.t.} \quad & \mathbf{C}'_n x = \bar{c} \\ & x \in \mathcal{X} \end{aligned} \tag{13.8}$$

where $\phi(x)$ is an objective function, opt is $\max$ when $\phi(x)$ is concave and $\min$ when $\phi(x)$ is convex, x is the column vector of portfolio weights, $\mathbf{C}_n$ is the centrality measure vector, $\bar{c}$ is the desired average centrality measure of the portfolio, and $\mathcal{X}$ is a convex set. To calculate the $\mathbf{C}_n$ we use the unweighted adjacency matrix of the MST or TMFG; this is the adjacency matrix that only considers the number of edges between two nodes.

Problem 13.8 is a general return-risk trade-off optimization problem; this means that we can consider the classic objective functions studied in Chap. 8. Also, the convex set $\mathcal{X}$ can include any of the constraints studied in Chap. 9 like linear constraints, constraints on convex risk measures, and tracking error constraints, among others.

Python code 13.10 shows how to implement the minimization of standard deviation with a constraint in the average degree of the portfolio model, and Fig. 13.17 shows the graphical representation of portfolio weights in the MST.

```

from scipy.linalg import sqrtm

# Define initial parameters
T, n = R.shape
Sigma = returns.cov().to_numpy()
Sigma_sqrt = sqrtm(Sigma)

# Node's degree based on MST and Pearson correlation
C_v = rp.centralty_vector(returns,
measure='Degree',
codependence="pearson",
graph="MST")

# Define Variables
x = cp.Variable((n,1))
g = cp.Variable(nonneg=True)

# Define risk measure
risk = g

# Define risk constraints
constraints = [cp.SOC(g, Sigma_sqrt @ x),
               C_v @ x == 1,
               cp.sum(x) == 1,
               x >= 0]

# Solve the problem
objective = cp.Minimize(risk)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_network_allocation(returns=returns,
                                w=weights,
                                codependence="pearson",
                                kind='kamada',
                                title='',
                                ax=ax)

```

Listing 13.10 Centrality measure constraint model code

13.1.7 Neighborhood Constraint

13.1.7.1 Mixed Integer Programming Approach

Ricca and Scozzari (2024) proposed an integer constraint that allows us to invest in assets that are not neighbors; this means that they are not directly linked through an edge. Cajas (2023c) generalized this idea using the range connection matrix $\mathbf{B}_{1,l}$ to assets that are not linked through walks of lengths that are lower or equal than l as follows:

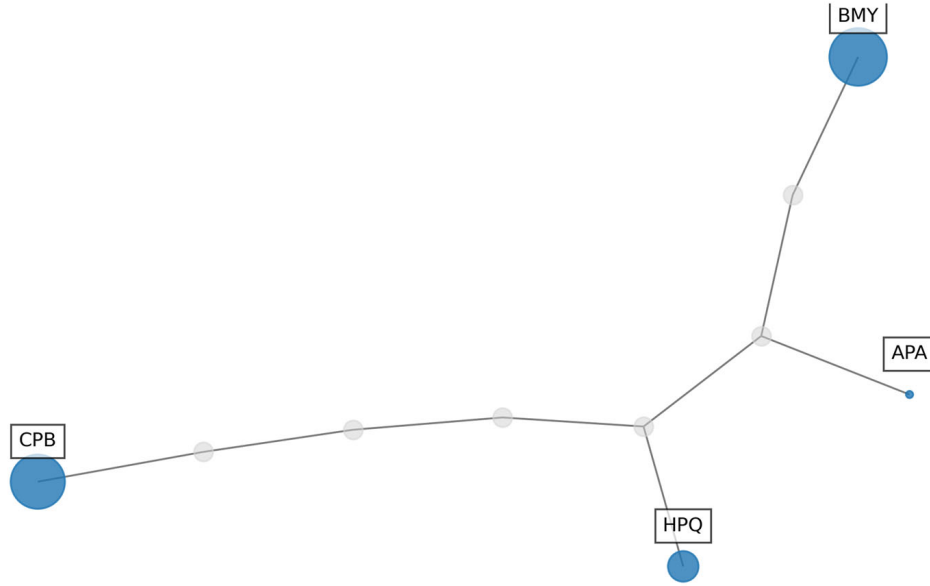


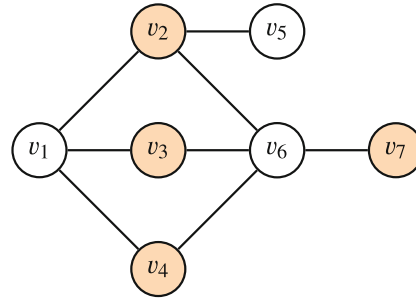
Fig. 13.17 Min variance with centrality constraint model asset allocation

$$\begin{aligned}
 \text{opt}_x \quad & \phi(x) \\
 \text{s.t.} \quad & (\mathbf{B}_{1,l} + I_n)z \leq 1 \\
 & x \leq uz \\
 & x \geq lz \\
 & z \in \{0, 1\}^n \\
 & x \in \mathcal{X}
 \end{aligned} \tag{13.9}$$

where z is an auxiliary binary vector variable of size $n \times 1$ that indicates if an asset is considered in the portfolio, I_n is the identity matrix of size n , and l and u are the lower and upper bounds of variable x . The constraint $(\mathbf{B}_{1,l} + I_n)z \leq 1$ is a special case of mutually exclusive investment constraint that was studied in Chap. 9. The Ricca and Scozzari model is the special case when $\mathbf{B}_{1,1} = A$.

To illustrate the idea of this model, if Fig. 13.18 is the graph that represents the relationships among seven assets. If we do not want to invest in assets that are connected by walks of length 1, this means $\mathbf{B}_{1,1} = A$; if we select asset v_1 we cannot invest in assets $\{v_2, v_3, v_4\}$ and also if we select asset v_6 we cannot invest in asset v_7 . The same idea can be extrapolated to assets that are connected by walks of lengths higher than 1 using the matrix $\mathbf{B}_{1,l}$.

Fig. 13.18 Example of graph of assets



The main advantage of this model is that it can be applied to any convex risk measure,⁹ and it can be combined easily with a cardinality constraint $\mathbf{1}'_n z \leq K$, where K is the maximum number of assets. The main disadvantages of this model are that it relies on a mixed integer programming (MIP) formulation, making it difficult to solve when we increase the number of assets, and that for matrices $(\mathbf{B}_{1,l} + I_n)$ that are near to the all-ones matrix, the problem cannot be solved.

Python code 13.11 shows how to implement the minimization of variance with mixed integer neighborhood constraint model and Fig. 13.19 shows the graphical representation of portfolio weights in the MST.

```

from scipy.linalg import sqrtm

# Define initial parameters
T, n = R.shape
Sigma = returns.cov().to_numpy()
Sigma_sqrt = sqrtm(Sigma)
I_n = np.identity(n)

# Connection matrix based on MST and Pearson correlation
# with walk size of length 1
B_1 = rp.connection_matrix(returns,
    codependence="pearson",
    graph="MST",
    walk_size=1)

# Define Variables
x = cp.Variable((n,1))
z = cp.Variable((n,1), boolean=True)
g = cp.Variable(nonneg=True)

# Define risk measure
risk = g

# Define risk constraints
constraints = [cp.SOC(g, Sigma_sqrt @ x),
    (B_1 + I_n) @ z <= 1,
    x <= z,

```

⁹ Currently MOSEK v10.2.1 does not support semidefinite and integer programming at the same time.

```

cp.sum(x) == 1,
x >= 0]

# Solve the problem
objective = cp.Minimize(risk)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_network_allocation(returns=returns,
    w=weights,
    codependence="pearson",
    kind='kamada',
    title='',
    ax=ax)

```

Listing 13.11 Min variance with mixed integer neighborhood constraint model code

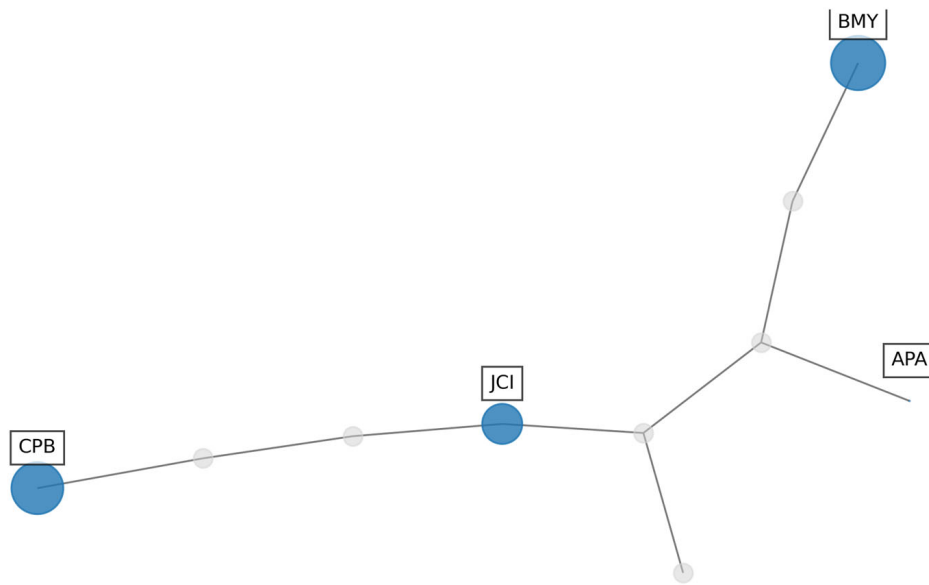


Fig. 13.19 Min variance with mixed integer neighborhood constraint model asset allocation

13.1.7.2 Semidefinite Programming Approach

Cajas (2023c) proposed a semidefinite programming (SDP) approach that allows us to invest in assets that are not linked through walks of lengths that are lower or equal than l . The idea is that if we want to avoid to invest in assets that are connected, the product of weights of the assets that are connected through connection matrix $\mathbf{B}_{1,l}$

approximates to 0. We can pose the minimization of variance with a constraint to invest in assets that are not linked through walks of length lower or equal than l as follows:

$$\begin{aligned}
 \min_{x, X} \quad & \text{Tr}(\Sigma X) \\
 \text{s.t.} \quad & \begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0 \\
 & \mathbf{B}_{1,l} \odot X = 0 \\
 & X \in \mathbf{S}^n \\
 & x \in \mathcal{X}
 \end{aligned} \tag{13.10}$$

where X is an auxiliary symmetric matrix variable of size $n \times n$ that approximates xx' through the Schur complement property that states that the constraint $\begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0$ is equivalent to $X - xx' \succeq 0$; $A \succeq 0$ means that the matrix A is positive semidefinite; Σ is the covariance matrix; $\mathbf{S}^n$ is the set of real symmetric matrices; and $\odot$ is the Hadamard product operator. Formulation 13.10 can be applied to risk measures whose optimization models can be expressed as SDP problems using the constraint $\begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0$ like kurtosis.

Python code 13.12 shows how to implement the minimization of variance with semidefinite neighborhood constraint model and Fig. 13.20 shows the graphical representation of portfolio weights in the MST.

```

# Define initial parameters
T, n = R.shape
Sigma = returns.cov().to_numpy()

# Connection matrix based on MST and Pearson correlation
# with walk size of length 1
B_1 = rp.connection_matrix(returns,
    codependence="pearson",
    graph="MST", # We can select the MST or TMFG
    walk_size=1)

# Define Variables
x = cp.Variable((n,1))
X = cp.Variable((n,n), symmetric=True)

M1 = cp.hstack([X, x])
M2 = cp.hstack([x.T, np.ones((1,1))])
M3 = cp.vstack([M1, M2])

# Define risk measure
risk = cp.trace(Sigma @ X)

# Define risk constraints
constraints = [M3 >> 0,

```

```

cp.multiply(B_1, X) == 0,
cp.sum(x) == 1,
x >= 0]

# Solve the problem
objective = cp.Minimize(risk * 1000)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_network_allocation(returns=returns,
                               w=weights,
                               codependence="pearson",
                               kind='kamada',
                               title='',
                               ax=ax)

```

Listing 13.12 Min variance with semidefinite neighborhood constraint model code

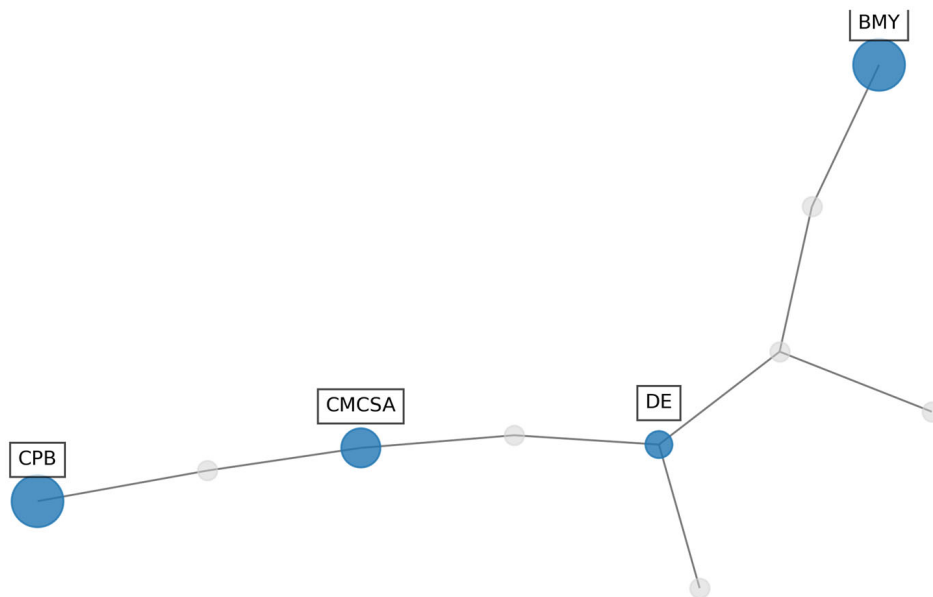


Fig. 13.20 Min variance with semidefinite neighborhood constraint model asset allocation

In the case of risk measures $\varrho(x)$ whose portfolio optimization models cannot be expressed using the SDP constraint, we can reduce the amount invested in assets that are connected adding a penalty function $\lambda \text{Tr}(X)$ in the objective function as follows:

$$\begin{aligned}
& \min_x \quad \varrho(x) + \lambda \text{Tr}(X) \\
& \text{s.t.} \quad \begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0 \\
& \quad \mathbf{B}_{1,l} \odot X = 0 \\
& \quad X \in \mathbf{S}^n \\
& \quad x \in \mathcal{X}
\end{aligned} \tag{13.11}$$

where λ is a penalty factor. The advantage of this approach compared to the MIP approach is that it gives us an approximate solution for matrices $\mathbf{B}_{1,l}$ that are near to the all-ones matrix when the MIP approach cannot find an optimal solution.

Python code 13.13 shows how to implement the minimization of CVaR with semidefinite neighborhood penalty model and Fig. 13.21 shows the graphical representation of portfolio weights in the MST.

```

# Define initial parameters
T, n = R.shape
alpha = 0.05

# Connection matrix based on MST and Pearson correlation
# with walk size of length 1
B_1 = rp.connection_matrix(returns,
                           codependence="pearson",
                           graph="MST", # We can select the MST or TMFG
                           walk_size=1)

# Define Variables
x = cp.Variable((n,1))
X = cp.Variable((n,n), symmetric=True)
t = cp.Variable((1, 1))
u = cp.Variable((T, 1))

M1 = cp.hstack([X, x])
M2 = cp.hstack([x.T, np.ones((1,1))])
M3 = cp.vstack([M1, M2])

# Define risk measure
risk = t + 1 / (alpha * T) * cp.sum(u)
penalty = 0.05 * cp.trace(X)

# Define risk constraints
constraints = [u >= -R @ x - t,
               u >= 0,
               M3 >> 0,
               cp.multiply(B_1, X) <= 0,
               cp.sum(x) == 1,
               x >= 0]

# Solve the problem
objective = cp.Minimize(risk + penalty)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

```

```
# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_network_allocation(returns=returns,
    w=weights,
    codependence="pearson",
    kind='kamada',
    title='',
    ax=ax)
```

Listing 13.13 Min CVaR with semidefinite neighborhood penalty model code

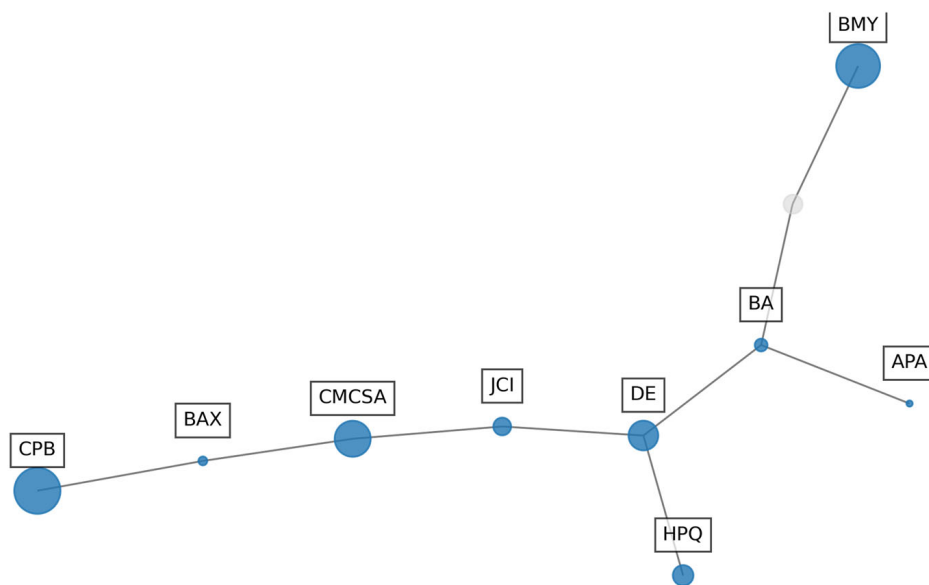


Fig. 13.21 Min CVaR with semidefinite neighborhood penalty model asset allocation

13.2 Cluster Constraint

13.2.1 Graphical Representations of Dendrograms

A dendrogram is a diagram that represents a tree. It is used in hierarchical clustering to illustrate the hierarchical relationships among variables. Also, it can be used to identify visually the clusters obtained through the hierarchical clustering process and an optimal number of clusters criterion.¹⁰

¹⁰ For more information about the hierarchical clustering and the optimal number of clusters see Chap. 12.

Python code 13.14 shows how to build Fig. 13.22 where we can see the dendrogram for a group of ten assets (APA, BA, BAX, BMY, CMCSA, CNP, CPB, DE, HPQ, JCI) using a hierarchical clustering algorithm based on Pearson correlation and Ward linkage; and the optimal number of clusters was determined using the two difference gap statistic.¹¹ On the other hand, using Python code 13.15 we can plot the clusters in Fig. 13.22 as two undirect complete graphs as shown in Fig. 13.23.

```
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
ax = rp.plot_dendrogram(returns=returns,
    codependence='pearson',
    linkage='ward',
    opt_k_method='twodiff',
    k=None,
    max_k=10,
    leaf_order=True,
    title='',
    ax=ax)
```

Listing 13.14 Dendrogram with Riskfolio-Lib

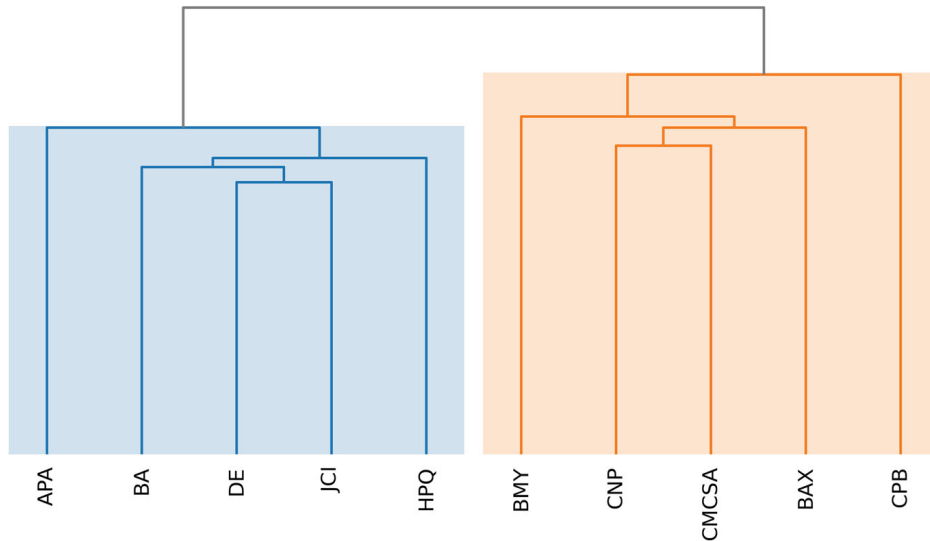


Fig. 13.22 Dendrogram of data sample based on Pearson correlation and Ward linkage

¹¹ For more information see Chap. 12 and Yue et al. (2008).

```
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
ax = rp.plot_clusters_network(returns=returns,
    codependence="pearson",
    linkage="ward",
    opt_k_method='twodiff',
    k=None,
    max_k=10,
    title='',
    ax=ax)
```

Listing 13.15 Dendrogram cluster network with Riskfolio-Lib

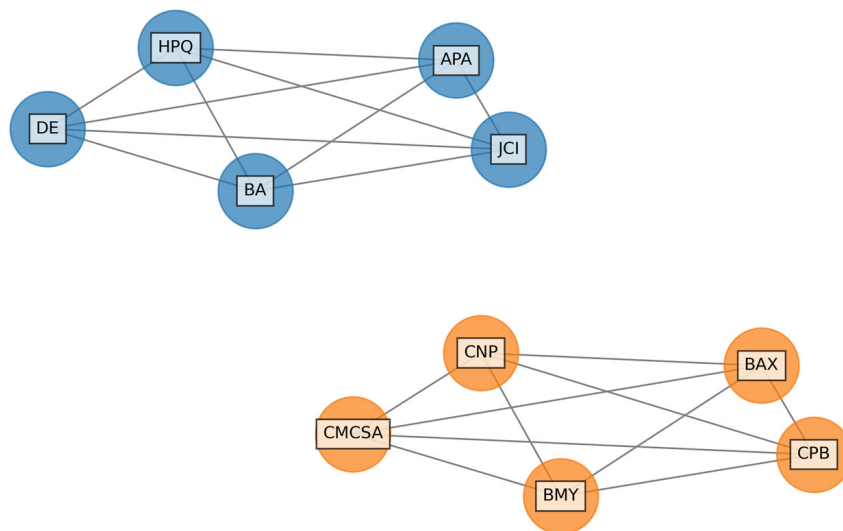


Fig. 13.23 Dendrogram clusters network of data sample

13.2.2 Matrix Representation of Dendrograms

The graphical representation allows us to have an initial idea of how assets are related; however, we cannot incorporate that information into the portfolio optimization process until we transform these graphs into a matrix form that allows us to build constraints.

The first matrix representation, which we call label matrix L , is based on the idea that each row represents a cluster (label) and each column represents an asset (variable), and in the intersection of each row and column we write a 1 if the asset belongs to that cluster or 0 if not. This kind of matrix is widely used in finance with traditional labels like industry, country, sector, etc. to build linear constraints

	APA	BA	BAX	BMV	CMCSA	CNP	CPB	DE	HPQ	JCI
Cluster 1	1	1	0	0	0	0	0	1	1	1
Cluster 2	0	0	1	1	1	1	1	0	0	0

Fig. 13.24 Label matrix L

related to limits on weights, for example, a concentration (max total weight) limit by industry.¹²

In Fig. 13.24 we can see the label matrix associated to clusters from dendrogram in Fig. 13.22. The second matrix representation, which we called adjacency label matrix L_A , is an adjacency matrix that represents the cluster network associated to the label matrix L . Mathematically, it is defined as:

$$L_A = L'L - I_n \quad (13.12)$$

where L is the label matrix, I_n is the identity matrix of size n , and n is the number of assets.

```
# Adjacency label matrix based on Pearson correlation
# and ward linkage
L_A = rp.clusters_matrix(returns,
    codependence='pearson',
    linkage='ward',
    opt_k_method='twodiff',
    k=None,
    max_k=10)
```

Listing 13.16 Adjacency label matrix code

Python code 13.16 shows how to build Fig. 13.25 where we can see the adjacency label matrix associated to Fig. 13.24. The adjacency label matrix L_A can be reordered as a diagonal block matrix where each block represents the adjacency matrix of the undirected complete graph of each cluster.

13.2.3 Dendrogram Measures for Portfolios

13.2.3.1 Percentage Invested in Related Assets

In the case of an investment portfolio, we can define the percentage invested in related assets in the same way as we define the percentage invested in connected assets of a portfolio in formula 13.7 but replacing the range connection matrix $\mathbf{B}_{1,I}$ with the adjacency label matrix L_A as follows:

¹² It is easy to see that the product Lx of the label matrix L and the vector of weights x gives the total weight per label.

	APA	BA	BAX	BMJ	CMCSA	CNP	CPB	DE	HPQ	JCI
APA	0	1	0	0	0	0	0	1	1	1
BA	1	0	0	0	0	0	0	1	1	1
BAX	0	0	0	1	1	1	1	0	0	0
BMJ	0	0	1	0	1	1	1	0	0	0
CMCSA	0	0	1	1	0	1	1	0	0	0
CNP	0	0	1	1	1	0	1	0	0	0
CPB	0	0	1	1	1	1	0	0	0	0
DE	1	1	0	0	0	0	0	0	1	1
HPQ	1	1	0	0	0	0	0	1	0	1
JCI	1	1	0	0	0	0	0	1	1	0

Fig. 13.25 Adjacency label matrix L_A

$$\mathbf{RA}(x) = \frac{\mathbf{1}'_n (L_A \odot |xx'|) \mathbf{1}_n}{\mathbf{1}'_n |xx'| \mathbf{1}_n} \quad (13.13)$$

Formula 13.13 allows us to measure the percentage invested in assets that belong to the same cluster. The diversification idea is to invest in assets in different clusters because if we invest in assets in the same cluster we are concentrating the risk in assets with a similar behavior; this means that for assets in the same clusters, the absolute value of the product of weights is 0 ($|x_i x_j| = 0$) because one of the assets must have a weight equal to 0 ($x_i = 0$ or $x_j = 0$).

Python code 13.17 shows how to calculate the percentage invested in related assets of the equally weighted portfolio.

```
# Equally weighted portfolio
T, n = R.shape
weights = np.ones((n,1))/n

# Percentage Invested in Connected Assets based on MST
# and Pearson correlation with walk size of length 1
CA = rp.connected_assets(returns,
    w=weights,
    codependence="pearson",
    graph="MST", # We can select the MST or TMFG
    walk_size=1)
```

Listing 13.17 Percentage invested in related assets with Riskfolio-Lib

13.2.4 Cluster Constraints

13.2.4.1 Mixed Integer Programming Approach

Cajas (2023d) proposed an integer programming constraint that allows us to invest only in assets that belong to different clusters. Using the label matrix L and the MIP formulation from problem 13.9, we can incorporate the information from clusters into the optimization process as follows:

$$\begin{aligned}
 \text{opt}_x \quad & \phi(x) \\
 \text{s.t.} \quad & Lz \leq 1 \\
 & x \leq uz \\
 & x \geq lz \\
 & z \in \{0, 1\}^n \\
 & x \in \mathcal{X}
 \end{aligned} \tag{13.14}$$

where z is an auxiliary binary vector variable of size $n \times 1$ that indicates if an asset is considered in the portfolio; and l and u are the lower and upper bounds of variable x . The idea of this model is that the mutually exclusive investment constraint $Lz \leq 1$ guarantees that we invest in one asset per cluster. We can generalize this constraint to $Lz \leq K$ where K is the maximum number of assets per cluster or even define a maximum number of asset per each cluster $L_i z \leq K_i$, where L_i is the i -th row of matrix L and K_i is the maximum number of assets of the i -th cluster. On the other hand, the constraint $Lz \leq 1$ works like a cardinality constraint where the maximum number of assets is the number of clusters.

The main advantage of this model is that it can be applied to any convex risk measure.¹³ The main disadvantage of this model is that it relies on a MIP formulation, which makes it difficult to solve when we increase the number of assets.

Python code 13.18 shows how to implement the minimization of variance with mixed integer cluster constraint model and Fig. 13.26 shows the graphical representation of portfolio weights in the cluster network graph.

```

from scipy.linalg import sqrtm

# Define initial parameters
T, n = R.shape
Sigma = returns.cov().to_numpy()
Sigma_sqrt = sqrtm(Sigma)
I_n = np.identity(n)

```

¹³ Currently MOSEK v10.2.1 does not support semidefinite and integer programming at the same time.

```

# Adjacency label matrix based on Pearson
# correlation and ward linkage
L_A = rp.clusters_matrix(returns,
    codependence='pearson',
    linkage='ward',
    opt_k_method='twodiff',
    k=None,
    max_k=10)
# Label Matrix
L = np.unique(L_A + I_n, axis=0)[::-1]

# Define Variables
x = cp.Variable((n,1))
z = cp.Variable((n,1), boolean=True)
g = cp.Variable(nonneg=True)

# Define risk measure
risk = g

# Define risk constraints
constraints = [cp.SOC(g, Sigma_sqrt @ x),
    L @ z <= 1,
    x <= z,
    cp.sum(x) == 1,
    x >= 0]

# Solve the problem
objective = cp.Minimize(risk)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_clusters_network_allocation(returns=returns,
    w=weights,
    codependence="pearson",
    linkage="ward",
    opt_k_method='twodiff',
    k=None,
    max_k=10,
    title='',
    ax=ax)

```

Listing 13.18 Min variance with mixed integer cluster constraint model code

13.2.4.2 Semidefinite Programming Approach

Cajas (2023d) showed that we can use the SDP approach of the neighborhood constraint to implement a constraint that allows us to invest in one asset per cluster. To implement this constraint, we only need to replace the range connection matrix $\mathbf{B}_{1,l}$ with the adjacency label matrix L_A in problem 13.10. The idea is that if we invest only in one asset per cluster, the product of weights of the assets that belong

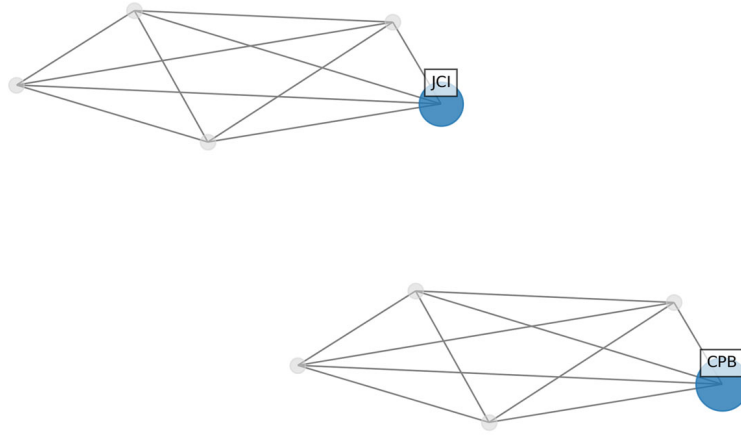


Fig. 13.26 Min variance with mixed integer cluster constraint model asset allocation

to the same cluster approximates to 0. We can pose the minimization of variance with a constraint to avoid investing in assets that are in the same cluster as follows:

$$\begin{aligned}
 \min_{x, X} \quad & \text{Tr}(\Sigma X) \\
 \text{s.t.} \quad & \begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0 \\
 & L_A \odot X = 0 \\
 & X \in \mathbf{S}^n \\
 & x \in \mathcal{X}
 \end{aligned} \tag{13.15}$$

In the same way as formulation 13.10, formulation 13.15 can be applied to risk measures whose optimization models can be expressed as SDP problems using the constraint $\begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0$ like kurtosis.

Python code 13.19 shows how to implement the minimization of variance with semidefinite cluster constraint model and Fig. 13.27 shows the graphical representation of portfolio weights in the cluster network graph.

```

# Define initial parameters
T, n = R.shape
Sigma = returns.cov().to_numpy()

# Adjacency label matrix based on Pearson
# correlation and ward linkage
L_A = rp.clusters_matrix(returns,
    codependence='pearson',
    linkage='ward',
    opt_k_method='twodiff',

```

```

    k=None,
    max_k=10)

# Define Variables
x = cp.Variable((n,1))
X = cp.Variable((n,n), symmetric=True)

M1 = cp.hstack([X, x])
M2 = cp.hstack([x.T, np.ones((1,1))])
M3 = cp.vstack([M1, M2])

# Define risk measure
risk = cp.trace(Sigma @ X)

# Define risk constraints
constraints = [M3 >> 0,
               cp.multiply(L_A, X) == 0,
               cp.sum(x) == 1,
               x >= 0]

# Solve the problem
objective = cp.Minimize(risk * 1000)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_clusters_network_allocation(returns=returns,
    w=weights,
    codependence="pearson",
    linkage="ward",
    opt_k_method='twodiff',
    k=None,
    max_k=10,
    title='',
    ax=ax)

```

Listing 13.19 Min variance with semidefinite cluster constraint model code

In the case of risk measures $\varrho(x)$ whose portfolio optimization models cannot be expressed using the SDP constraint, we can replace the range connection matrix $\mathbf{B}_{1,I}$ with the adjacency label matrix L_A in formulation 13.11 as follows:

$$\begin{aligned}
 \min_x \quad & \varrho(x) + \lambda \text{Tr}(X) \\
 \text{s.t.} \quad & \begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0 \\
 & L_A \odot X = 0 \\
 & X \in \mathbf{S}^n \\
 & x \in \mathcal{X}
 \end{aligned} \tag{13.16}$$

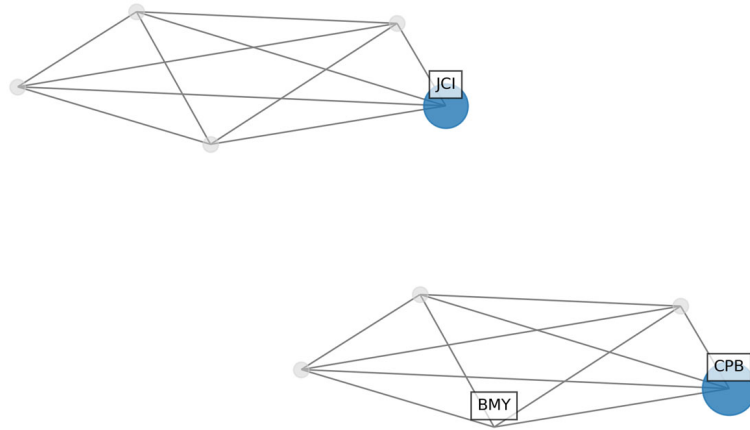


Fig. 13.27 Min variance with semidefinite cluster constraint model asset allocation

The advantage of the SDP approach compared to the MIP approach is that it gives us an approximate solution even when the MIP model cannot find a solution. The main disadvantage of the SDP approach is that we cannot generalize this constraint to invest in more than one asset per cluster like in the case of the MIP approach. Finally, this constraint also works like an approximate cardinality constraint where the maximum number of assets is the number of clusters.

Python code 13.20 shows how to implement the minimization of CVaR with semidefinite cluster penalty model and Fig. 13.28 shows the graphical representation of portfolio weights in the cluster network graph.

```
# Define initial parameters
T, n = R.shape
alpha = 0.05

# Adjacency label matrix based on Pearson
# correlation and ward linkage
L_A = rp.clusters_matrix(returns,
    codependence='pearson',
    linkage='ward',
    opt_k_method='twodiff',
    k=None,
    max_k=10)

# Define Variables
x = cp.Variable((n,1))
X = cp.Variable((n,n), symmetric=True)
t = cp.Variable((1, 1))
u = cp.Variable((T, 1))

M1 = cp.hstack([X, x])
M2 = cp.hstack([x.T, np.ones((1,1))])
M3 = cp.vstack([M1, M2])

# Define risk measure
risk = t + 1 / (alpha * T) * cp.sum(u)
```

```

penalty = 0.05 * cp.trace(X)

# Define risk constraints
constraints = [u >= -R @ x - t,
              u >= 0,
              M3 >> 0,
              cp.multiply(L_A, X) <= 0,
              cp.sum(x) == 1,
              x >= 0]

# Solve the problem
objective = cp.Minimize(risk + penalty)
prob = cp.Problem(objective, constraints)
prob.solve('MOSEK')

# Plot Asset Allocation
fig, ax = plt.subplots(figsize=(8,5), tight_layout=True)
weights = pd.DataFrame(x.value, index=assets)
ax = rp.plot_clusters_network_allocation(returns=returns,
                                       w=weights,
                                       codependence="pearson",
                                       linkage="ward",
                                       opt_k_method='twodiff',
                                       k=None,
                                       max_k=10,
                                       title='',
                                       ax=ax)

```

Listing 13.20 Min CVaR with semidefinite cluster penalty model code

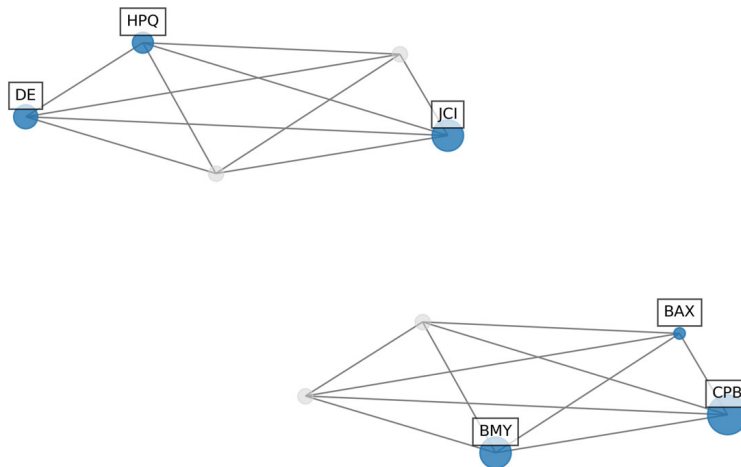


Fig. 13.28 Min CVaR with semidefinite cluster penalty model asset allocation